ENGS 3302 PATTERN RECOGNITION · STUDENT NOTES

Week 2: Features, Distance and the Nearest-Mean Classifier

Sources: Duda, Hart & Stork, *Pattern Classification*, 2nd ed., §1.3, §2.6.1, §4.6, §10.6.1 · ISLP §2.2 · Palmer Penguins data: Horst, Hill & Gorman (2020), CC0 · course lecture notes

What you must be able to do after this week (exam scope)

1. Give examples of features extracted from images, sounds and text, and tell numerical features from categorical ones.
2. Compute the Euclidean and the Manhattan distance between two feature vectors.
3. Explain why the units of a feature can change a distance-based decision.
4. Normalize features with min-max scaling and with the z-score, using numbers from the training data.
5. Explain why a classifier is measured on test data it has not seen, and compute its accuracy and error rate.
6. Classify a new sample with the nearest-mean classifier, by hand and in Python.
7. Say when the nearest-mean classifier works well and when it fails.

Not required: the formal properties of a metric, the Minkowski family beyond $k = 1, 2$, and the Tanimoto distance (Duda §4.6).

Before you start: read the *Week 2 Math Toolkit* (vectors, distance, mean, standard deviation) and try the Math check on Moodle.
Reading (optional, these notes are complete): Duda §4.6 (pp. 187–188) on distances and scaling.

2.1 Features: turning objects into numbers

[Duda §1.3, pp. 11–12]

A classifier never sees a fish, a face or a word. It sees numbers. **Feature extraction** is the stage that turns raw data into a few useful numbers, the **features**. In Week 1 the camera gave a photo of a fish, and we kept two numbers: its *lightness* and its *width*.

Raw data	Example features	Type
Image of a fish	mean brightness of the fish pixels, width, length	numerical
Image of a digit	how much ink, how many strokes, ink in the top half	numerical
Sound recording	loudness, pitch, duration	numerical
E-mail	how many times “free” appears, number of links	numerical (counts)
Customer record	city, payment method	**categorical**

A **numerical** feature is a number where size has a meaning: 20 cm is wider than 16 cm. A **categorical** feature is a label from a list, such as *Gaza, Khan Younis, Rafah*. Distances need numbers, so categorical features must first be coded as numbers; the lab shows how (*one-hot encoding*: one 0/1 column per category).

What makes a good feature? (from Week 1)

Similar values for samples of the same class, different values for samples of different classes. A feature should also be cheap to measure and not just repeat another feature.

2.2 Feature vectors and distance

[Duda §4.6, p. 187]

With d features, each sample is a **feature vector** $\mathbf{x} = (x_1, x_2, \ldots, x_d)$: a point in a d-dimensional **feature space**. Fish have $d = 2$; the Iris flowers in the lab have $d = 4$. Many classifiers decide by asking *which known samples or class centres are closest?* So we need a way to measure distance.

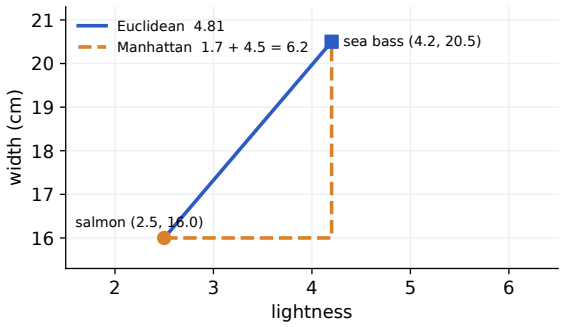
Two distances between a and b

Euclidean: $\|\mathbf{a} - \mathbf{b}\| = \sqrt{\sum_{i=1}^{d} (a_i - b_i)^2}$ Manhattan: $\sum_{i=1}^{d} |a_i - b_i|$

The **Euclidean distance** is the straight line between the points. The **Manhattan distance** (also called *city-block*) walks along the axes, like a car in a city with a grid of streets.

Worked example 2.1

Salmon $\mathbf{a} = (2.5, 16.0)$, sea bass $\mathbf{b} = (4.2, 20.5)$.
Differences: -1.7 and -4.5.
Euclidean: $\sqrt{1.7^2 + 4.5^2} = \sqrt{23.14} \approx \mathbf{4.81}$.
Manhattan: $1.7 + 4.5 = \mathbf{6.2}$.




```
import numpy as np
a = np.array([2.5, 16.0]);  b = np.array([4.2, 20.5])
np.linalg.norm(a - b)       # Euclidean: 4.81
np.sum(np.abs(a - b))       # Manhattan: 6.2
```

Which one to use? Euclidean is the default and is what we use unless we say otherwise. Manhattan gives less weight to one very large difference, because it does not square it.

2.3 When the units decide

[Duda §4.6, Fig. 4.18, p. 188]

A distance only sees numbers. It does not know that one feature is in centimetres and another is on a 0–10 scale. Change the units of one feature and the distances change, and so can the decision.

Worked example 2.2 — same fish, different units

The ten fish of Week 1 have these class centres (means), with width in cm: salmon (3.88, 17.36), sea bass (5.94, 20.00). A new fish has lightness 7.0 and width 18.0 cm.

Width in cm: distance to salmon centre $\sqrt{3.12^2 + 0.64^2} \approx 3.18$; to sea bass centre $\sqrt{1.06^2 + 2.00^2} \approx 2.26$. $\Rightarrow$ **sea bass**.

Width in mm (multiply widths by 10): centres (3.88, 173.6) and (5.94, 200.0), new fish

(7.0, 180). Distances $\sqrt{3.12^2 + 6.4^2} \approx 7.12$ and $\sqrt{1.06^2 + 20^2} \approx 20.03$. $\Rightarrow$ **salmon**.

The fish did not change. Only the unit of one feature did, and the answer flipped: in mm, width differences are ten times larger and take over the distance.

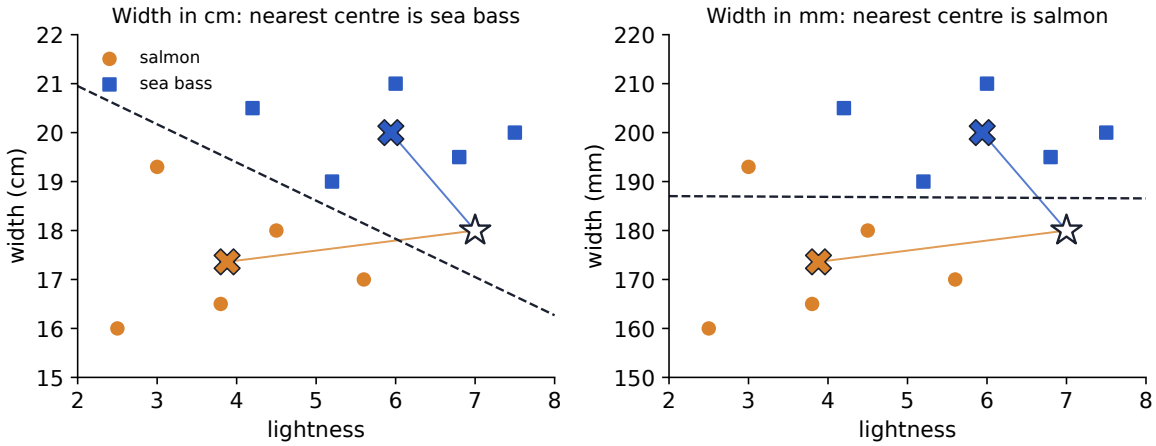


Figure 1: The same ten fish (dots and squares), class centres (crosses) and new fish (star). Dashed line: the nearest-mean boundary. Changing width from cm to mm moves the boundary and changes the decision.

Be careful

Never compare raw features with very different ranges in a distance. First bring them to a common scale. This is called **normalization** (or *feature scaling*).

2.4 Normalization

[Duda §10.6.1, p. 538]

Normalization rescales each feature separately, so that all features have similar ranges. Two methods are used almost everywhere.

Min-max scaling and z-score

For each feature, using the *training* data:

$$\text{min-max: } x' = \frac{x - \min}{\max - \min} \quad \text{(result between 0 and 1)}$$

$$\text{z-score: } z = \frac{x - \bar{x}}{\sigma} \quad \text{(result has mean 0 and standard deviation 1)}$$

Worked example 2.3 — normalizing the new fish

Training data (the ten fish), lightness: min 2.5, max 7.5, mean 4.91, standard deviation 1.529. Width (cm): min 16.0, max 21.0, mean 18.68, standard deviation 1.638.

Min-max: lightness $\frac{7.0-2.5}{7.5-2.5} = 0.9$; width $\frac{18.0-16.0}{21.0-16.0} = 0.4$.

z-score: lightness $\frac{7.0-4.91}{1.529} \approx 1.37$; width $\frac{18.0-18.68}{1.638} \approx -0.42$.

After either method the nearest centre is **sea bass**, and the answer is the same whether width was measured in cm or in mm: the z-score of 180 mm is exactly the z-score of 18.0 cm.

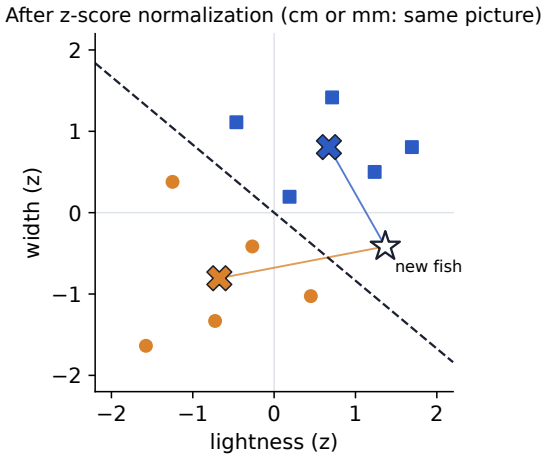


Figure 2: The ten fish after z-score normalization. Both features now have mean 0 and standard deviation 1.

Use the training numbers for everything

Compute the mean, standard deviation, min and max *once*, from the training data. Then apply the *same* numbers to every new sample. Recomputing them on new data, or on all data including the samples kept aside for testing (§2.6), gives the classifier information it should not have. This mistake is called **data leakage**.

Which one? The z-score is the usual default. Min-max is natural when a feature has fixed limits, such as pixel values from 0 to 255; but one extreme value can squeeze all the others together.

2.5 The nearest-mean classifier

[Duda §2.6.1, pp. 36–37]

Now we can build our first classifier. Each class is represented by one point, its **mean** (centre, or *prototype*). A new sample goes to the class with the nearest mean.

Nearest-mean classifier (also: minimum-distance classifier)

Train: for each class ω_i, compute the mean vector $\boldsymbol{\mu}_i$ of its training samples.
Classify: compute the distance $\|\mathbf{x} - \boldsymbol{\mu}_i\|$ to every class mean and choose the smallest.

Worked example 2.4 — ten fish, one new fish

Train. Salmon lightness: $(2.5 + 3.0 + 3.8 + 4.5 + 5.6)/5 = 3.88$; width: $(16.0 + 19.3 + 16.5 + 18.0 + 17.0)/5 = 17.36$. So $\boldsymbol{\mu}_{\text{salmon}} = (3.88,\ 17.36)$. In the same way $\boldsymbol{\mu}_{\text{sea bass}} = (5.94,\ 20.00)$.

Classify $\mathbf{x} = (7.0,\ 18.0)$: distances 3.18 to salmon and 2.26 to sea bass (Worked example 2.2) $\Rightarrow$ **sea bass**.

Check on the training fish: all ten are on the correct side, so 0 training errors. (With width in mm, one salmon, $(3.0,\ 19.3)$, would be misclassified.)

The boundary is a straight line. The points at equal distance from two means form the line (in d dimensions, the plane) that cuts the segment between the means in half, at a right angle. That is the dashed line in Figures 1 and 2. Duda shows that this simple rule is the best possible rule when the classes are equally likely and each class is a round cloud of the same size around its mean; we

meet this result again in Week 5.

When it fails. One centre per class is a good summary only if each class is *one compact group*. In Figure 3, class A is two separate groups; its mean falls in the middle, right inside class B, and the classifier misclassifies 29 of the 75 points.

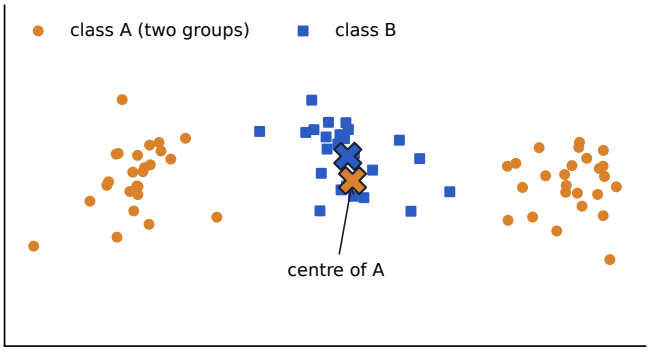


Figure 3: When a class is made of separate groups, its mean can be far from all its samples.

2.6 How good is the classifier? Training data and test data

[ISLP §2.2.1–2.2.2]

We now have a classifier. How do we know whether it is any good?

We build a classifier for *new* samples, not for the ones it learned from. So we cannot judge it on its own training data: a classifier that simply memorized the training samples would score 100% on them and tell us nothing about new ones. Before training, we therefore split the labelled data in two parts:

- **training data** (often 70%): used to learn everything, including the normalization numbers and the class centres;
- **test data** (often 30%): kept aside, unseen until the end, and used once to measure the finished classifier.

Accuracy and error rate, on the test data

$$\text{accuracy} = \frac{\text{number of correct decisions}}{\text{number of test samples}} \qquad \text{error rate} = 1 - \text{accuracy}$$

Example — Penguins, from the lab

The Palmer Penguins data has 342 penguins of 3 species, each with four measurements: three in millimetres and the body mass in grams. A 30% test split keeps 103 penguins aside and trains on the other 239. The nearest-mean classifier with z-score normalization gets 100 of the 103 test penguins right: accuracy $100/103 \approx 0.97$, error rate $3/103 \approx 0.03$. Without normalization it gets 65 right: accuracy $65/103 \approx 0.63$.

With only ten fish we do not split: ten samples are too few to give a trustworthy test score. In Python the split is one line. `stratify=y` keeps the share of each class the same in both parts, and `random_state` fixes the random split, so everyone gets the same numbers:

```
from sklearn.model_selection import train_test_split
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=0, stratify=y)
```

Is normalizing always better? Now we can answer with test data. No. It helps when ranges differ for reasons unrelated to the classes, such as units (in the lab, the penguins mix millimetres with grams, and nearest-mean accuracy rises from 63% to 97%). When all features already share one unit and the large ranges carry information, as with the Iris flowers (all in cm), it can even hurt (91% falls to 76%). Always compare on test data.

Week 6 returns to evaluation in depth: which mistakes the classifier makes, and how to test with small data.

2.7 In Python

By hand with NumPy, then the same model in scikit-learn:

```
import numpy as np
X = np.array([[2.5,16.0],[3.0,19.3],[3.8,16.5],[4.5,18.0],[5.6,17.0],
              [4.2,20.5],[5.2,19.0],[6.0,21.0],[6.8,19.5],[7.5,20.0]])
y = np.array(["salmon"]*5 + ["sea bass"]*5)
new = np.array([7.0, 18.0])

centres = {c: X[y == c].mean(axis=0) for c in ["salmon", "sea bass"]}
dists   = {c: np.linalg.norm(new - m) for c, m in centres.items()}
print(dists)                      # salmon 3.18, sea bass 2.26
print(min(dists, key=dists.get)) # sea bass
```

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestCentroid

model = make_pipeline(StandardScaler(), NearestCentroid())
model.fit(X, y)                   # learns mean/std, then the class centres
model.predict([[7.0, 18.0]])      # array(['sea bass'])
```

The pipeline learns the normalization numbers from the training data in `fit`, and reuses them in `predict`: exactly the rule in the box on data leakage.

Summary

- Features turn raw data into numbers; categorical features need coding before distances can use them.
- Euclidean distance: square, add, square root. Manhattan distance: add the absolute differences.
- Distances depend on units. Normalize first: min-max to $[0, 1]$, or the z-score to mean 0 and standard deviation 1.
- Judge a classifier on test data it has never seen: accuracy = correct / number of test samples.
- Compute normalization numbers on the training data only, and reuse them for new data.
- Nearest-mean classifier: one mean per class, choose the nearest. Its boundary is a straight line; it fails when a class is made of separate groups.

Check yourself

1. A new fish has lightness 5.0 and width 19.0 cm. Using the class means in Worked example 2.4 and Euclidean distance, which class does the nearest-mean classifier choose?
2. Repeat question 1 with the Manhattan distance. Is the answer the same?
3. For the same fish, compute its z-scores using the training numbers in Worked example 2.3.
4. A colleague normalizes the training data and the test data separately, each with its own mean and standard deviation. What is wrong, and why?
5. Give an example from your own life of a class that is made of two separate groups. Why would a nearest-mean classifier struggle with it?
6. A classifier gets 94 of 103 test samples right. What are its accuracy and its error rate? Why is its accuracy on the training data not a fair measure?

Answers: 1. sea bass (1.99 vs 1.37). 2. sea bass (2.76 vs 1.94). 3. 0.06 and 0.20. 4. data leakage: the test data must use the training numbers. 6. 0.913 and 0.087; the classifier has already seen the training data.